



Integrantes: Anguaya Angel, Chicaiza Andrea
Fecha: 20/04/2026

Taller Grupal

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo Nativo Puro (Swift/Kotlin)

Surgió por la necesidad de exprimir al máximo el hardware. Al principio, era la única opción para garantizar fluidez.

- **Cómo funcionan:** Utilizan los lenguajes y APIs oficiales (Swift/Objective-C para iOS; Kotlin/Java para Android). El código se compila directamente a instrucciones que el procesador entiende.
- **Herramientas:** Xcode y Android Studio.
- **Relevancia:** Alta. Siguen siendo el estándar de oro para aplicaciones que requieren alto rendimiento (juegos, edición de video) o integración profunda con el sistema. Son la base sobre la que se construyen todas las demás herramientas.

2. La Era del WebView / Híbridos (Cordova, Ionic inicial)

Surgieron para permitir que los desarrolladores web crearan apps sin aprender lenguajes móviles, buscando "escribir una vez y ejecutar en todos lados".

- **Cómo funcionan:** La app es básicamente un navegador web (WebView) "disfrazado" de aplicación nativa. El contenido es HTML/CSS/JS.
- **Herramientas:** Cordova, PhoneGap, Ionic (versiones iniciales).
- **Relevancia:** Baja. Han perdido terreno frente a opciones más potentes. Se mantienen en apps corporativas simples o prototipos rápidos, pero sufren de falta de fluidez ("jank") y problemas de diseño que no se siente "natural" al sistema.

3. El Puente Nativo-JavaScript (React Native)

Apareció como el "punto medio" ideal: la flexibilidad de JavaScript con componentes que se ven y sienten nativos.

- **Cómo funcionan:** No usan un WebView. El código JS se comunica con los componentes nativos a través de un "Bridge" (Puente). Tú escribes un botón en React y el puente le dice al sistema que renderice un UIButton real en iOS.
- **Herramientas:** React Native, Appcelerator Titanium.

- **Relevancia:** Alta. Es la opción preferida por empresas que necesitan iterar rápido (Meta, Shopify). Aunque el "puente" puede ser un cuello de botella, nuevas arquitecturas (como JSI en React Native) están eliminando esa lentitud.

4. El Motor de Renderizado Propio (Flutter)

La evolución más reciente y radical. En lugar de pedirle al sistema sus botones, la herramienta los dibuja ella misma.

- **Cómo funcionan:** Funcionan como un motor de videojuegos (similar a Unity). Flutter no usa componentes del sistema; dibuja cada píxel en un lienzo (Canvas) usando el motor Skia (o Impeller). Esto garantiza que la app se vea exactamente igual en cualquier dispositivo.
- **Herramientas:** Flutter (Dart).
- **Relevancia:** Alta. Es extremadamente popular por su velocidad de desarrollo y rendimiento cercano al nativo. Es ideal para interfaces personalizadas y complejas.

5. El Futuro: Foldables, AR y Wearables

- **Foldables (Plegables):** Han obligado a las herramientas (especialmente Flutter y Android Nativo) a adoptar el Diseño Adaptativo. Ya no basta con "Responsive"; la app debe detectar cambios físicos en la pantalla en tiempo real.
- **Realidad Aumentada (AR):** Aquí el Nativo Puro sigue mandando. Aunque existen plugins para Flutter/React Native, el control preciso de la cámara y los sensores que requieren ARKit (iOS) y ARCore (Android) hace que el desarrollo nativo sea la opción lógica para experiencias inmersivas serias.
- **Wearables:** El desarrollo es cada vez más específico. Herramientas como SwiftUI o Jetpack Compose son fundamentales aquí por su ligereza y capacidad de adaptarse a pantallas circulares o diminutas.

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

Caso de Estudio A: Shopify (Construido con React Native)

Caso de Estudio B: eBay (Construido con Flutter)

Instrucciones para el equipo: Deben debatir, investigar (pueden usar sus equipos) y redactar un reporte crítico respondiendo las siguientes interrogantes. No busquen respuestas "correctas", busquen respuestas fundamentadas con ingeniería.

1. Ingeniería Inversa de la Decisión:

- ¿Por qué Shopify, una plataforma cuyo núcleo es el comercio electrónico y la gestión de inventarios masivos, decidió apostar por React Native para unificar sus aplicaciones tras años de desarrollo nativo independiente?

La decisión se basó en la velocidad de iteración y consistencia lógica. Shopify ya tenía una base sólida en React para web; apostar por React Native permitió que sus desarrolladores front-end contribuyeran al móvil sin una curva de aprendizaje masiva. Al ser el núcleo del negocio la gestión de datos e inventarios, React Native permite compartir la compleja lógica de negocio entre plataformas mientras mantiene una UI que se siente nativa.

- ¿Por qué eBay, que tiene una infraestructura técnica heredada de décadas, decidió migrar hacia Flutter para permitir que sus ingenieros compartieran casi el 100% del código entre iOS y Android sin perder la calidad visual?

eBay necesitaba eliminar la disparidad de funciones. Con el desarrollo nativo tradicional, una funcionalidad nueva tardaba meses en igualarse entre iOS y Android. Flutter permitió a eBay alcanzar un 98.3% de código compartido, asegurando que la experiencia visual (crucial para un marketplace de subastas) fuera idéntica y de alta fidelidad sin importar el sistema operativo.

2. Análisis de Rendimiento y Arquitectura:

- Considerando que Shopify depende de un ecosistema de terceros, ¿qué estrategias de Native Modules o Fabric (la nueva arquitectura) creen que tuvo que priorizar para integrar periféricos de hardware como lectores de tarjetas y escáneres de inventario sin sacrificar la velocidad de la UI?

Shopify priorizó la migración a Fabric (la nueva arquitectura) para eliminar el "Bridge" tradicional. Al usar JSI (JavaScript Interface), la aplicación puede invocar métodos nativos de forma síncrona. Esto es vital para periféricos como escáneres de inventario: la respuesta debe ser inmediata para que el usuario no sienta lag entre el escaneo físico y la actualización en pantalla.

- ¿Cómo utiliza eBay el motor de renderizado de Flutter para manejar la alta carga de imágenes y micro-interacciones en su catálogo global, evitando que la experiencia se sienta lenta en dispositivos de gama media?

eBay aprovecha que Flutter dibuja sus propios píxeles usando Skia/Impeller. Para dispositivos de gama media, esto evita el costo de CPU de "traducir" widgets a componentes del sistema. Al manejar miles de imágenes, Flutter utiliza un sistema de caché de imágenes eficiente y un ciclo de vida de renderizado que prioriza los 60 FPS, evitando que las micro-interacciones de las subastas se sientan lentas.

3. El Desafío de las Nuevas Pantallas:

- Si mañana el estándar de compra en Shopify se traslada a lentes de Realidad Aumentada, ¿podría React Native adaptarse rápidamente al ser un puente hacia componentes nativos, o su arquitectura de "puente" causaría demasiada latencia para una experiencia inmersiva fluida?

Su capacidad de adaptación es alta porque actúa como un pegamento (host). Podría integrarse con ARKit (iOS) o ARCore (Android) mediante módulos nativos. El riesgo es la latencia: si la lógica de la experiencia AR reside demasiado en el hilo de JavaScript, el retraso en la comunicación podría causar mareos (motion sickness) o falta de fluidez en la superposición de objetos.

- Ante un cambio hacia interfaces de Realidad Aumentada en eBay, ¿qué ventaja le daría a Flutter su motor propio para dibujar objetos 3D personalizados, o fallaría al no poder integrarse directamente con el hardware gráfico de bajo nivel?

La ventaja es su control total sobre el canvas. Flutter podría renderizar capas de interfaz de usuario 3D personalizadas con extrema rapidez. Sin embargo, su debilidad es que no tiene acceso directo a las APIs de bajo nivel del hardware gráfico sin pasar por canales de plataforma (Platform Channels), lo que complica la implementación de sombreadores (shaders) complejos en tiempo real comparado con el acceso directo nativo.

4. Veredicto Crítico:

- Como futuros ingenieros de software: Si estuvieran creando hoy una aplicación que requiere interactuar con mapas 3D interactivos, notificaciones de alta frecuencia y debe soportar pantallas plegables... ¿Elegirían Nativo, React Native o Flutter? Fundamenten su respuesta basados estrictamente en la arquitectura de la herramienta, no en gustos personales por un lenguaje.

Elección: Nativo

Mapas 3D: Requieren una gestión de memoria intensa y acceso directo a la GPU sin capas de abstracción que degraden el rendimiento.

Notificaciones de alta frecuencia: El manejo de hilos (multithreading) y servicios en segundo plano es más robusto y menos propenso a ser terminado por el sistema operativo en nativo.

Pantallas Plegables: Las APIs de continuidad de pantalla (como Jetpack WindowManager) se lanzan primero para nativo; las soluciones multiplataforma suelen tardar meses en implementar adaptaciones estables para hardware emergente.

Pregunta final:

¿Qué es un aplicativo nativo, hay varios, como se clasifican?

Un aplicativo nativo (o aplicación nativa) es un programa de software diseñado y desarrollado exclusivamente para un sistema operativo móvil específico, ya sea iOS (Apple) o Android (Google). Estas aplicaciones se construyen utilizando lenguajes de programación y herramientas de desarrollo propias de cada plataforma (SDKs), lo que les permite integrarse perfectamente con el hardware y software del dispositivo.

Clasificación de aplicativos nativos:

1. Por plataforma de destino

- **iOS Nativo:** desarrollado con Swift u Objective-C usando Xcode y los SDKs de Apple (UIKit, SwiftUI, ARKit, etc.)
- **Android Nativo:** desarrollado con Kotlin o Java usando Android Studio y los SDKs de Google (Jetpack Compose, ARCore, etc.)

2. Por tipo de dispositivo

- **Móvil** (smartphones)
- **Wearables** (Apple Watch, Wear OS): requieren frameworks ligeros como SwiftUI o Jetpack Compose por las limitaciones de pantalla y recursos
- **Tablets y Foldables:** exigen diseño adaptativo para manejar cambios físicos de pantalla en tiempo real

3. Por framework de UI nativo

- **UIKit / SwiftUI:** ecosistema Apple
- **XML Layouts / Jetpack Compose:** ecosistema Android

Link de la presentación: <https://taller-analisis-critico--x2de3aq.gamma.site/>